

COURSE CONTENTS



Course Agenda

Module - Lua Programming
Language.

Beginner to Advanced

Adding 
Value

Table of Contents: Lua Programming Language.

NOTE: Depth and breadth of Modules/Sections covered depend on participants and program level, objectives, and duration of the course.

Objectives

Upon completion of this course, you will be able to:

- Understand of Lua Language Design
- Understand Object Oriented Programming in Lua
- Working with Lua
- Programming in Lua
- Lua Programming Language Design and Best Practices

Audience

This course is designed for beginners as well as developers interested in learning Lua Programming Language.

Prerequisites

Participants *must* know

Concepts of Object Oriented Programming Programming in C and Java Language

Energy and Openness to Learn, Attitude to Take Ownership Appreciation and Openness to Design Thinking Understanding and Appreciation of Mathematics 10+2 Level Strong Logical and Programming Ability

Following Training Schedule and Decorum: Arriving on Time and Avoiding Distractions

Software/Hardware Requirements

Hardware: Computer/Laptop with 8GB+ RAM and Minimum Core i5+ Processor

Operating System: Ubuntu 24.04, 64 Bit or Windows 10+ 64 Bit or Mac OSX SDL: Lua Compiler

IDE: Sublime Text/Visual Studio Code/Vim/Any Editor and IntelliJ IDE Others: Internet/Whiteboard/Marker etc.

Training Pedagogy, Methodology and Conventions

Training pedagogy, methodology, mechanisms and training techniques are decided by the trainer based on many factors including duration, agenda, objectives, topics, trainees diversity, meeting prerequisites etc. He will be the sole decision maker in terms of how topics to be covered and prioritization of topics. Training objectives are to build competency of trainees in required know-how to face programming challenges. Solutions building/specific problems solving are generally part of consultancy rather than training, Specific examples are covered as proof of concepts rather than full solutions. Please note this training is not designed for any specific learner, rather for class as a whole.

Lua Programming Language

Program Structure **

The Kotlin Training program is structured in the following course.

ESSENTIAL COURSES

COURSES	COURSE TITLE	RECOMMENDED DISTRIBUTION **
COURSE A1	Lua Programming Language Advanced Lua	05 to 06 Full Days
COURSE A2	Programming Language	06 to 07 Full Days

1. 8 Hours/Full Day Training session duration consists of 1 hour lunch break, 15 minutes two tea/coffee breaks. Hence «ffgcĖivg Training Hours ™gr Full DaD will bg 6 and 5 Hours after excluding lunch and tea/coffee breaks from Full Day training session duration.

1. Depth and breadth of Modules/Sections covered depends on participants and program level, objectives, duration of the course, participants diversity, prior knowledge and participants meeting the prerequisites, logical and aptitude, diversity of batch etc. and total time available during the training. Every batch is different in terms of diversity, experience, and prior knowledge etc. hence can affect estimates.

2. This training agenda is quite extensive. Trainer will prioritize topics to reach around 95+

% coverage on best effort basis depending on participants diversity, prior knowledge and unlearning/learning/grasping capabilities, time available during the training etc.

3. Generally on the average we cover 2 Modules per full day session. ™lgasg noĖg ĖhaĖ fiĖĖing morg numbgrs of Modulgs pgr sgssion and/or rgducing sgssion Ėimg and Ėraining duraĖion, will impacĖ gffgcĖivngngss and qualiĖD of Ėhg Ėraining and can lgad Ėo rgducĖion in inĖgracĖions, hand on Ėimg %, Ėimg for qugrigs/qugsĖions and in dgpĖh covragg and discussions gĖc. In Ėhis scgnario Ėhg Ėraingr will ĖrD Ėo complgĖg Ėopics bD prioriĖizing on Ėhg bgsĖ gfforĖ basis and rgquirg parĖicipanĖs coopgraĖion in Ėgrms of Ėafing owngrship in guidgd lgarning and following guidglings and insĖrucĖions givgn bD Ėhg Ėraingr.
-

Lua Programming Language

Detailed Course Outline

COURSE A1: LUA PROGRAMMING LANGUAGE

Module 01: Lua Getting Started

- Chunks
- Some Lexical Conventions Global Variables
- Types and Values Nil Booleans
- The Stand-Alone Interpreter

Module 02: Numbers

- Numerals
- Arithmetic Operators Relational Operators
- The Mathematical Library Random-number generator Rounding functions
- Representation Limits Conversions Precedence
- Lua Before Integers

Module 03: Strings

- Literal strings Long strings Coercions
- The String Library Unicode

Module 04: Tables

- Table Indices Table Constructors
- Arrays, Lists, and Sequences Table Traversal
- Safe Navigation The Table Library

Module 05: Functions

- Multiple Results Variadic Functions
- The Function Table.unpack Proper Tail Calls

Module 06: The External World

- The Simple I/O Model
- The Complete I/O Model
- Other Operations on Files
- Other System Calls
- Running system commands

Module 07: Miscellaneous Topics

- Local Variables and Blocks Control Structures if then else while repeat
- Numerical for Generic for break, return, and goto

Module 08: Closures

- Functions as First-Class Values Non-Global Functions
- Lexical Scoping
- A Taste of Functional Programming

Module 09: Pattern Matching

- The Pattern-Matching Functions The Function String.find The Function String.match The Function String.gsub The Function String.gmatch
- Patterns Captures Replacements
- URL encoding Tab expansion
- Tricks of the Trade

Module 10: Date and Time

- TheFunctionos.time
- TheFunctionos.date
- Date–Time Manipulation

Module 11: Bits and Bytes

- Bitwise Operators
- Unsigned Integers
- Packing and Unpacking Binary Data
- Binary files

Module 12: Data Structures

- Arrays
- Matrices and Multi-Dimensional Arrays Linked Lists
- Queues and Double-Ended Queues
- Reverse Tables Sets and Bags String Buffers Graphs

Module 13: Data Files and Serialization

- Data Files Serialization
- Saving tables without cycles Saving tables with cycles

Module 14: Compilation, Execution, and Errors

- Compilation
- Precompiled
- Code
- Errors
- Error Handling and Exceptions
- Error Messages and Tracebacks

COURSE A2: ADVANCED LUA PROGRAMMING LANGUAGE

Module 15: Modules and Packages

- The Function require
- Renaming a module Path searching Searchers
- The Basic Approach for Writing Modules in Lua Submodules and Packages

Module 16: Iterators and the Generic for

- Iterators and Closures
- The Semantics of the Generic for
- Stateless Iterators
- Traversing Tables in Order
- True Iterators

Module 17: Metatables and Metamethods

- Arithmetic Metamethods
- Relational Metamethods
- Library-Defined Metamethods
- Table-Access Metamethods
- The `_index` metamethod
- The `_newindex` metamethod
- Tables with default values Tracking table accesses
- Read-only tables

Module 18: Object-Oriented Programming

- Classes Inheritance
- Multiple Inheritance Privacy
- The Single-Method Approach Dual Representation

Module 19: The Environment

- Global Variables with Dynamic Names Global-Variable Declarations
- Non-Global Environments Using `_ENV`
- Environments and Modules
- `_ENV` and `load`

Module 20: Garbage

- Weak Tables Memorize Functions Object Attributes
- Revisiting Tables with Default Values Ephemeron Tables
- Finalizers
- The Garbage Collector Controlling the Pace of Collection

Module 21: Coroutines

- Coroutine Basics Who Is the Boss?
- Coroutines as Iterators Event-Driven Programming

Module 22: Reflection

- Introspective Facilities
- Accessing local variables
- Accessing non-local variables
- Accessing other coroutines
- Hooks
- Profiles
- Sandboxing

Module 23: An Overview of the C API

- A First Example
- The Stack
- Pushing elements
- Querying elements
- Other stack operations
- Error Handling with the C API
- Error handling in application code
- Error handling in library code
- Memory Allocation

Module 24: Extending Your Application

- The Basics
- Table Manipulation
- Calling Lua Functions
- A Generic Call Function

Module 25: Calling

- C from Lua
- C Functions
- Continuations
- C Modules

Module 26: Techniques for Writing C Functions

- Array Manipulation
- String Manipulation
- Storing State in C Functions
- The registry Upvalues
- Shared upvalues

Module 27: User-Deffined Types in C

- Userdata Metatables
- Object-Oriented Access
- Array Access
- Light Userdata

Module 28: Managing Resources

- A Directory Iterator
- An XML Parser

Module 29: Threads and States

- Multiple Threads
- Lua States

Module 30: Lua Unit Testing

- LuaUnit Introduction
- Writing Tests
- Grouping Tests setup/teardown
- Using Command Line